

Teoria della dimostrazione, Teoria dei tipi, Proof Assistants e AI

verso una nuova pratica della matematica

Marino Miculan

Scuola Estiva di Logica AILA 2026

Bardonecchia, 31 agosto – 5 settembre 2026

DMIF, Università di Udine | marino.miculan@uniud.it | <https://marino.miculan.org>

LEZIONE 1

Logica intuizionista e isomorfismo di Curry–Howard: il λ -calcolo semplicemente tipato

AILA 2026 — Bardonecchia

Obiettivo

Far percepire «proposizioni come tipi, dimostrazioni come programmi» non come uno slogan, ma come una corrispondenza **precisa** e **verificabile a mano**.

1. Lezione 1 — Curry–Howard

- 1.1 Che cosa vuol dire «controllare» una dimostrazione
- 1.2 Logica intuizionista in quindici minuti
- 1.3 L'isomorfismo di Curry–Howard
- 1.4 Sintassi e giudizi
- 1.5 L'esempio canonico
- 1.6 Estensioni: prodotti, somme, unità, vuoto

Blocco 1

Da Hilbert a Brouwer, da Brouwer a Curry–Howard

Abbiamo visto *perché* serve  *Introduzione: crisi della complessità, Kepler, Liquid Tensor*.
Ora la domanda tecnica: **come** è possibile?

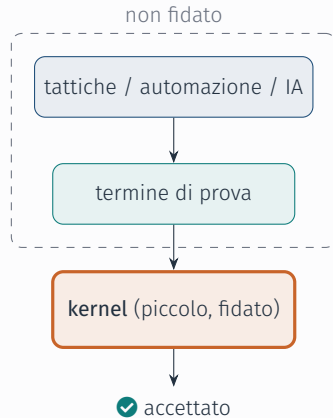
La domanda di fondo

Se una dimostrazione è un *oggetto matematico*, allora «essere una dimostrazione di A » è una proprietà *decidibile*. Chi la decide? E perché dovremmo fidarci?

Il de Bruijn criterion

Un sistema soddisfa il **de Bruijn criterion** se produce oggetti-prova controllabili da un **kernel** piccolo, indipendente e ispezionabile.

- ▶ la fiducia si concentra su poche migliaia di righe di codice;
- ▶ è possibile avere più implementazioni del kernel, e confrontarle;
- ▶ tattiche, automazione, IA *non* sono nella base fidata: qualunque cosa producano, deve passare dal kernel;
- ▶ 🔗 L6: *è esattamente ciò che rende sensato lasciar scrivere dimostrazioni a un modello di IA*



Una genealogia in 60 anni

Anno	Sistema	L'idea che ha introdotto
1967	Automath	le dimostrazioni sono termini, e una macchina li controlla
1979	LCF / ML	il tipo astratto thm : tattiche libere, correttezza garantita
1987	Edinburgh LF	un metalinguaggio unico (λII) per definire qualunque logica
1989	Coq $\rightarrow$ Rocq	CIC: Prop impredicativo, tipi induttivi, estrazione di programmi
1994	Isabelle/HOL	HOL classica, prove leggibili (Isar), automazione (Sledgehammer)
2007	Agda 2	Martin-Löf Type Theory predicativa: programmare e dimostrare sono la stessa cosa
2013	Lean	metaprogrammazione nel linguaggio stesso (Lean 4, 2021), e Mathlib

Tutti i sistemi citati soddisfano il *de Bruijn criterion*.

Tutti, tranne Isabelle/HOL, si basano sull'isomorfismo di Curry–Howard e sulla logica intuizionista.

Perché rifiutare il terzo escluso

Classicamente

$$A \vee \neg A \quad \neg\neg A \rightarrow A$$

Una dimostrazione può stabilire $\exists x. P(x)$ senza esibire alcun x .

Intuizionisticamente

$$A \vee \neg A \quad \times$$

Dimostrare $\exists x. P(x)$ significa **costruire** un testimone.

Per il pubblico di filosofia

Non è (solo) prudenza tecnica: è una tesi sul significato. Il significato di un enunciato è dato dalle sue condizioni di *dimostrazione*, non dalle sue condizioni di verità (Dummett, anti-realismo).

Brouwer–Heyting–Kolmogorov: *che cosa è* una dimostrazione?

- ▶ una dimostrazione di $A \wedge B$ **è** una coppia $\langle p, q \rangle$: p dimostra A , q dimostra B ;
- ▶ una dimostrazione di $A \vee B$ **è** una scelta di uno dei due disgiunti, più una sua dimostrazione;
- ▶ una dimostrazione di $A \rightarrow B$ **è** un *metodo* che trasforma dimostrazioni di A in dimostrazioni di B ;
- ▶ non esiste alcuna dimostrazione di $\perp$; e $\neg A := A \rightarrow \perp$.

Intuizione

Questa è già, informalmente, **propositions as types**: la clausola BHK di ogni connettivo descrive la *struttura dei dati* dei suoi testimoni.

La tabella di corrispondenza

Logica	Tipi	Programmi
proposizione A	tipo A	specifica
dimostrazione di A	termine $t : A$	programma
$A \rightarrow B$	$A \rightarrow B$ (funzioni)	fun $x \Rightarrow \dots$
$A \wedge B$	$A \times B$ (prodotto)	coppia
$A \vee B$	$A + B$ (somma disgiunta)	unione etichettata
$\top$	1 (unità)	()
$\perp$	0 (tipo vuoto)	—
$\neg A$	$A \rightarrow \mathbf{0}$	—
normalizzazione	β -riduzione	esecuzione

Quantificatori $\forall/\exists$:  L2–L3: System F e tipi dipendenti

ALWAYS HAS BEEN

**WAIT, PROVING
IS JUST PROGRAMMING?**

Blocco 2

Il λ -calcolo semplicemente tipato

Simply Typed Lambda Calculus (STLC): sintassi

Tipi $A, B ::= \iota \mid A \rightarrow B$
Termini $t, u ::= x \mid \lambda x:A. t \mid t u$
Contesti $\Gamma ::= \diamond \mid \Gamma, x : A$

- ▶ i contesti sono liste di ipotesi con variabili *distinte*;
- ▶ il giudizio fondamentale è $\Gamma \vdash t : A$: «nel contesto Γ , il termine t ha tipo A »;
- ▶ letto logicamente: «dalle ipotesi Γ segue A , e t ne è la dimostrazione».

$$\frac{(x : A) \in \Gamma}{\Gamma \vdash x : A} \text{VAR}$$

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x:A. t : A \rightarrow B} \rightarrow I$$

$$\frac{\Gamma \vdash t : A \rightarrow B \quad \Gamma \vdash u : A}{\Gamma \vdash t u : B} \rightarrow E$$

Cancellando i termini...

...si ottengono *esattamente* le regole della deduzione naturale per il frammento implicativo della logica intuizionista. $\rightarrow I$ è la scarica dell'ipotesi; $\rightarrow E$ è il *modus ponens*.

$$\frac{}{(\lambda x:A. t) u \longrightarrow_{\beta} t[x := u]} \beta \qquad \frac{t \longrightarrow_{\beta} t'}{t u \longrightarrow_{\beta} t' u} \text{APP}_{\ell}$$

- ▶ **Subject reduction:** se $\Gamma \vdash t : A$ e $t \longrightarrow_{\beta} t'$ allora $\Gamma \vdash t' : A$;
- ▶ **Church–Rosser:** $\twoheadrightarrow_{\beta}$ è confluyente;
- ▶ **Strong Normalization:** ogni riduzione da un termine tipabile termina.

Le dimostrazioni degli ultimi due sono lunghe e non le faremo qui: Girard–Lafont–Taylor, capp. 4 e 6.

⚠ Consequenza logica

SN + confluenza $\Rightarrow$ forma normale unica $\Rightarrow$ coerenza: nessun t chiuso ha tipo $\perp$. *La consistenza è una proprietà computazionale.*

Subject reduction: schema della dimostrazione

Se $\Gamma \vdash t : A$ e $t \longrightarrow_{\beta} t'$, allora $\Gamma \vdash t' : A$.

IL LEMMA CHIAVE — SOSTITUZIONE

$$\Gamma, x:A \vdash u : B \text{ e } \Gamma \vdash v : A \implies \Gamma \vdash u[x := v] : B$$

per induzione sulla derivazione di $\Gamma, x:A \vdash u : B$; il caso VAR si spezza in x e $y \neq x$.

IL CASO INTERESSANTE — IL REDESSO

$\Gamma \vdash (\lambda x:A. u) v : B$	ipotesi
$\implies \Gamma \vdash \lambda x:A. u : A \rightarrow B \text{ e } \Gamma \vdash v : A$	inversione di $\rightarrow E$
$\implies \Gamma, x:A \vdash u : B$	inversione di $\rightarrow I$
$\implies \Gamma \vdash u[x := v] : B$	lemma di sostituzione

Gli altri casi sono le *congruenze*: ipotesi induttiva, e si riapplica la stessa regola.

Esempio di derivazione: Transitività dell'implicazione

Costruiamo alla lavagna il termine di tipo $(A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow (A \rightarrow C)$.

$$\frac{\frac{\frac{\frac{\frac{\Gamma \vdash f : A \rightarrow B}{\Gamma \vdash g : B \rightarrow C}}{\Gamma \vdash g (f a) : C} \rightarrow E}{f:A \rightarrow B, g:B \rightarrow C \vdash \lambda a:A. g (f a) : A \rightarrow C} \rightarrow I}{f:A \rightarrow B \vdash \lambda g. \lambda a. g (f a) : (B \rightarrow C) \rightarrow A \rightarrow C} \rightarrow I}{\vdash \lambda f. \lambda g. \lambda a. g (f a) : (A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow A \rightarrow C} \rightarrow I$$

Con $\Gamma = f:A \rightarrow B, g:B \rightarrow C, a:A$.

 L4: ritroveremo questo termine identico in Lean: **fun** a => h2 (h1 a)

Congiunzione: il prodotto $A \times B$

SINTASSI

$$A, B ::= \dots \mid A \times B \quad t, u ::= \dots \mid \langle t, u \rangle \mid \pi_1 t \mid \pi_2 t$$

TIPAGGIO

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash u : B}{\Gamma \vdash \langle t, u \rangle : A \times B} \times I$$

$$\frac{\Gamma \vdash t : A \times B}{\Gamma \vdash \pi_1 t : A} \times E_1$$

$$\frac{\Gamma \vdash t : A \times B}{\Gamma \vdash \pi_2 t : B} \times E_2$$

β -RIDUZIONE

$$\pi_1 \langle t, u \rangle \longrightarrow_{\beta} t$$

$$\pi_2 \langle t, u \rangle \longrightarrow_{\beta} u$$

Il costruttore è $\wedge I$, i distruttori sono $\wedge E$: ridurre cancella una *deviazione*.

Disgiunzione: la somma $A + B$

SINTASSI

$$\begin{array}{l} A, B ::= \dots \mid A + B \\ t, u ::= \dots \mid \text{in}_1 t \mid \text{in}_2 t \mid \text{case } t \text{ of } [\text{in}_1 x \Rightarrow u \mid \text{in}_2 y \Rightarrow v] \end{array}$$

TIPAGGIO

$$\frac{\Gamma \vdash t : A_i}{\Gamma \vdash \text{in}_i t : A_1 + A_2} +I_i \qquad \frac{\Gamma \vdash t : A + B \quad \Gamma, x:A \vdash u : C \quad \Gamma, y:B \vdash v : C}{\Gamma \vdash \text{case } t \text{ of } [\text{in}_1 x \Rightarrow u \mid \text{in}_2 y \Rightarrow v] : C} +E$$

β -RIDUZIONE

$$\begin{array}{l} \text{case } (\text{in}_1 t) \text{ of } [\text{in}_1 x \Rightarrow u \mid \text{in}_2 y \Rightarrow v] \longrightarrow_{\beta} u[x := t] \\ \text{case } (\text{in}_2 t) \text{ of } [\text{in}_1 x \Rightarrow u \mid \text{in}_2 y \Rightarrow v] \longrightarrow_{\beta} v[y := t] \end{array}$$

Il **case** è $\vee E$: per usare $A \vee B$ devo saper concludere C in *entrambi* i casi.

1 — IL VERO

$A ::= \dots \mid \mathbf{1} \quad t ::= \dots \mid \langle \rangle$

$$\frac{}{\Gamma \vdash \langle \rangle : \mathbf{1}} \mathbf{1I}$$

Un costruttore, **nessun** distruttore: niente da eliminare, nessuna β -riduzione.

0 — IL FALSO

$A ::= \dots \mid \mathbf{0} \quad t ::= \dots \mid \text{abort}_A t$

$$\frac{\Gamma \vdash t : \mathbf{0}}{\Gamma \vdash \text{abort}_A t : A} \mathbf{0E}$$

Un distruttore, **nessun** costruttore: niente da introdurre, di nuovo nessuna β -riduzione.

⚠ L'asimmetria che conta

$\mathbf{0E}$ è l'*ex falso quodlibet*, e $\neg A := A \rightarrow \mathbf{0}$. Che $\mathbf{0}$ non abbia *alcuna* regola di introduzione è ciò da cui, fra un attimo, ricaveremo la coerenza dell'intero sistema.

Le proprietà si conservano — e da qui la coerenza

Subject reduction, Church–Rosser e strong normalization valgono ancora per $\lambda_{\rightarrow}$ esteso con $\times$, $+$, **1** e **0**.

Le dimostrazioni si estendono caso per caso: ogni nuova β -regola aggiunge un caso, e nulla di più.

FORME CANONICHE

se $\vdash t : A$ t normale è...

$A \rightarrow B$ $\lambda x:A. t'$

$A \times B$ $\langle t_1, t_2 \rangle$

$A + B$ $\text{in}_1 t' \text{ o } \text{in}_2 t'$

1 $\langle \rangle$

0 **niente**

- ▶ per **SN** ogni termine chiuso ha una forma normale;
- ▶ per **CR** quella forma normale è unica;
- ▶ una forma normale *chiusa* è sempre un'introduzione del proprio tipo.

⚠ Coerenza

Nessun termine chiuso ha tipo **0**, mentre **1** ne ha esattamente uno: il sistema distingue il falso dal vero, cioè $\perp$ non è dimostrabile.

Non tutti i tipi sono abitati

Lo stesso argomento non vale solo per **0**. Dato un tipo, cercarne un abitante *normale* è una ricerca con un numero *finito* di scelte: se fallisce, il tipo è **vuoto** — cioè la proposizione non è un teorema.

ABITATI

$A \rightarrow A$ $\lambda x. x$

$A \rightarrow B \rightarrow A$ $\lambda x. \lambda y. x$

$A \rightarrow \neg\neg A$ $\lambda x. \lambda k. k x$

VUOTI

$A + \neg A$

$\neg\neg A \rightarrow A$

$((A \rightarrow B) \rightarrow A) \rightarrow A$

terzo escluso

doppia negazione

legge di Peirce

Tutti e tre a destra sono tautologie *classiche*: basta la tavola di verità.

Il confine con la logica classica

I tre schemi sono equivalenti fra loro sopra la logica intuizionista: aggiungerne uno qualunque dà esattamente la logica classica.  L4: *lo faremo in Lean — ed è coerente*

Perché la legge di Peirce è vuota

Cerchiamo un t chiuso di tipo $((A \rightarrow B) \rightarrow A) \rightarrow A$. Per SN e CR basta guardare fra i **normali**.

1. il tipo è una freccia $\Rightarrow t = \lambda k. t'$, con $k : (A \rightarrow B) \rightarrow A \vdash t' : A$;
2. A atomico $\Rightarrow t'$ non è un'introduzione: è k applicata a qualcosa, $t' = k u$ con $u : A \rightarrow B$;
3. ancora una freccia $\Rightarrow u = \lambda x:A. v$, con $k, x:A \vdash v : B$;
4. B atomico $\Rightarrow$ anche v è una variabile applicata: ma k produce A , e x non è una funzione.

Nessuna mossa.

La ricerca termina sempre: l'abitazione in $\lambda \rightarrow$ è *decidibile*, e PSPACE-completa [Sta79].

💡 E allora che cosa sarebbe un suo abitante?

Un operatore di controllo: Peirce è il tipo di **call/cc** [Gri90]. Curry–Howard arriva alla logica classica, ma il calcolo smette di essere puro.

Che cosa manca ancora?

- ▶ Abbiamo tutta la logica proposizionale intuizionista.
- ▶ **Non** abbiamo i quantificatori.
- ▶ Due strade, che percorreremo entrambe:
 - ▶ quantificare sui *tipi*: $\forall X. A$ — System F 🔗 L2: polimorfismo
 - ▶ quantificare sui *termini*: tipi dipendenti 🔗 L2-L3: λP , MLTT
- ▶ Il λ -cubo di Barendregt organizzerà queste direzioni in una mappa unica.

🔧 Da provare prima di L4

Costruire i termini di tipo $A \rightarrow \neg\neg A$ e $\neg\neg\neg A \rightarrow \neg A$.
Poi rifare per $\neg\neg A \rightarrow A$ l'argomento della legge di Peirce.

- ▶ M. H. Sørensen e P. Urzyczyn. *Lectures on the Curry–Howard Isomorphism*. Vol. 149. Studies in Logic and the Foundations of Mathematics. Elsevier, 2006
- ▶ J.-Y. Girard, Y. Lafont e P. Taylor. *Proofs and Types*. Vol. 7. Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 1989
- ▶ B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002 — capitoli 9 e 11.
- ▶ T. G. Griffin. «A Formulae-as-Types Notion of Control». In: *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '90)*. ACM, 1990, pp. 47–58 — il contenuto computazionale della logica classica.